



ENME 441

06 – Shift Registers

Convert serial input data to parallel output pins to control many outputs from just a few ESP32 pins

Topics



PART 1

motivation

Expand pin access

- Serial in, parallel out
- Input vs output registers

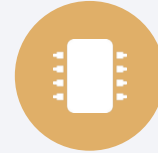


PART 2

shifting

One bit per clock pulse

- Shift vs storage register
- Eight pulses, then a latch
- Chaining



PART 3

hardware

SN74HC595

- Shift register pinout
- Wiring to the ESP32-C3



PART 4

code

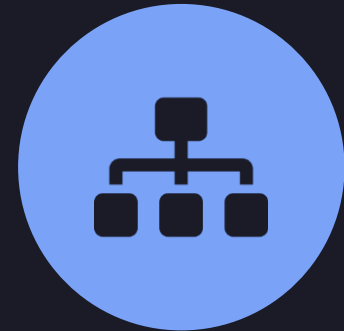
Driving through MicroPython

- Bit-banged Shifter class
- SPI Shifter class

01

Motivation

Expanded access to GPIO pins with higher output current



Shift Registers



Output shift register

`serial in, parallel out`

- Send bits one at a time from the ESP32
- Each bit is shifted to a different output pin
- Common 8-bit output SRs:
 - `SN74HC595` (35 mA per pin, about the same as the ESP32-C3)
 - `TPIC6B595` (150 mA per pin)



Input shift register

`parallel in, serial out`

- Load voltages from several input pins at once
- Shift them out one at a time to be read
- Common 8-bit input SR: TI SN74HC165

TRADING CLOCK CYCLES FOR GPIO PINS

3 GPIO pins plus 8 clock pulses, instead of 8 GPIO pins. You are spending time (a few tens of microseconds) to access GPIO pins (and output currents with the TPIC6B595) the C3 does not provide.



ENME 441 focuses on output only

We will use the SN74HC595, but all concepts apply directly to both higher-current output SRs (like the TPIC6B595) and input SRs (just flip the input/output direction).

Latching Output Shift Registers

Bits go in one at a time on clock signal, and outputs change simultaneously on latch signal.

1

Serial to parallel

A binary sequence arrives one bit at a time on a single GPIO pin and comes out as 8 simultaneous output levels.

2

Latching

The stages are loaded one by one, but nothing on the output pins moves until you pulse the latch. No glitching partial patterns.

3

Isolation from the load

The register drives the output circuit, not your GPIO pin. The C3 only ever sees logic-level signals.

4

Cheap to extend

Chain more SRs using the same 3 pins on the ESP32 to drive 16, 24, 32, ... outputs.

FOUR SIGNALS

- **Serial**
The data bit you are presenting right now.
- **Clock**
Shift every stage one place along and load Serial into the first one.
- **Latch**
Copy all eight stages to the output pins at once.
- **Output**
QA–QH, the eight parallel lines that drive your load.

02

Shifting process

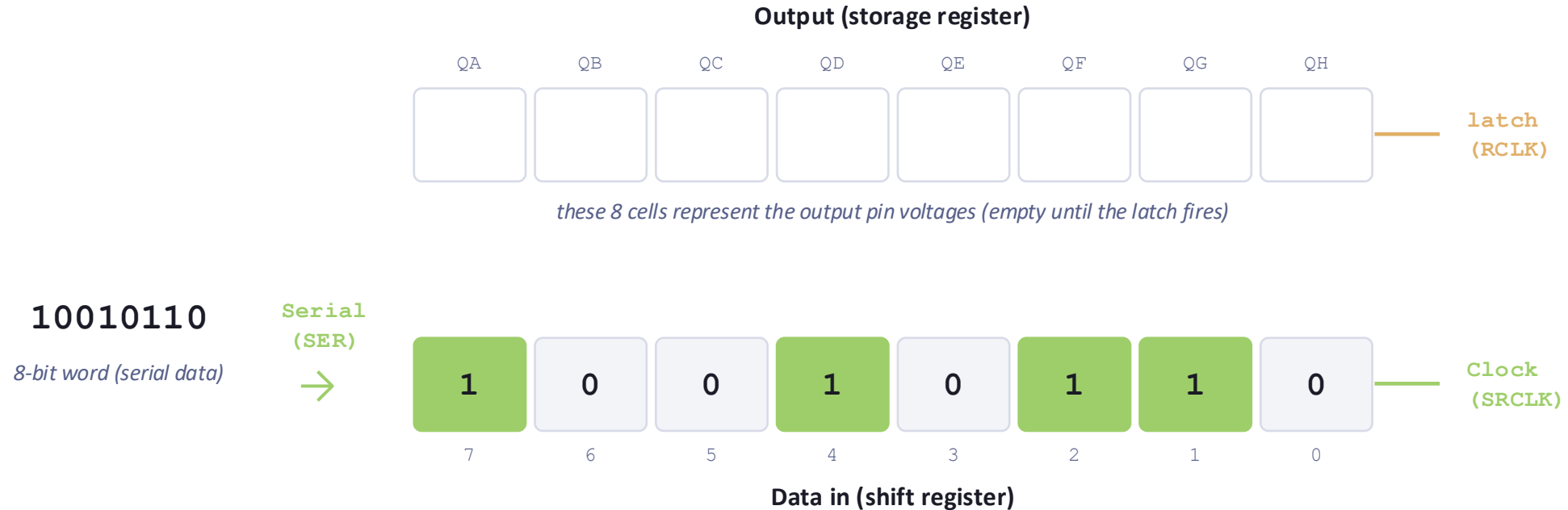
One bit per clock pulse



Two Registers in One Chip

A register is a small block of memory made from an array of flip-flops used to store a set of bits.

The SR contains 2 registers: the shift register holds bits as they are delivered, and the storage register holds the voltage outputs after latching.



- **Two registers, two clocks**

SRCLK moves data through the shift register. RCLK copies the shift register into the storage register. They are independent.

- **Nothing is on the output pins yet**

The word is fully loaded above, but the outputs (QA–QH) are initially floating (or hold whatever values were previously latched).

- **All bits enter @ MSB of the shift register**

New data lands in the first stage and walks right. The bit you send first ends up furthest along, at QH.

One Bit Per Clock Pulse

Each rising edge on the clock shifts every stage one place along and loads the next bit into QA.



- **The word is sent LSB first**

Present LSB first. The first bit out of the ESP32 travels the furthest into the register.
- **So QA ends up holding the MSB**

After eight pulses the register reads left to right exactly like the written word: QA is bit 7, QH is bit 0.
- **A 9th bit pushes the LSB out on QH'**

A bit clocked past stage H leaves on QH'. On a single chip it is discarded; in a chain it becomes the next chip's input.

Latching

Loading the shift register changes nothing at the outputs. Pulsing the latch (RCLK) changes all 8 outputs at once.



- **Both clocks are rising edge triggered**

SRCLK and RCLK act on a rising edge. Drive the pin high to trigger, wait a bit, then drive it low again to get ready for the next cycle.

- **The outputs move together**

Every output changes on the same edge. Without the latch the outputs would continually change as each bit arrives.

- **Reload while the outputs hold**

You can clock a whole new word into the shift register without touching RCLK.

Cascading: 16 Outputs, Same 3 Pins

QH' holds the bit falling off the end of one chip. Wire it to the next chip's serial input, with shared SRCLK and RCLK signals on both chips.



16 clock pulses, then 1 latch

Send all 16 bits, then pulse RCLK once. Both chips latch together, so all 16 outputs change at the same instant.

The first byte ends up in the last SR

The low byte is sent first and gets pushed all the way into SR #2.

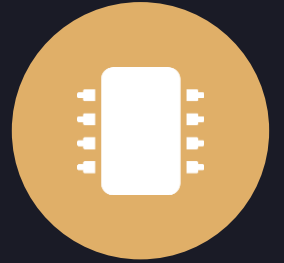
Keep going

3 chips is 24 outputs, 4 is 32, etc. The pin count never changes, only the number of clock pulses.

03

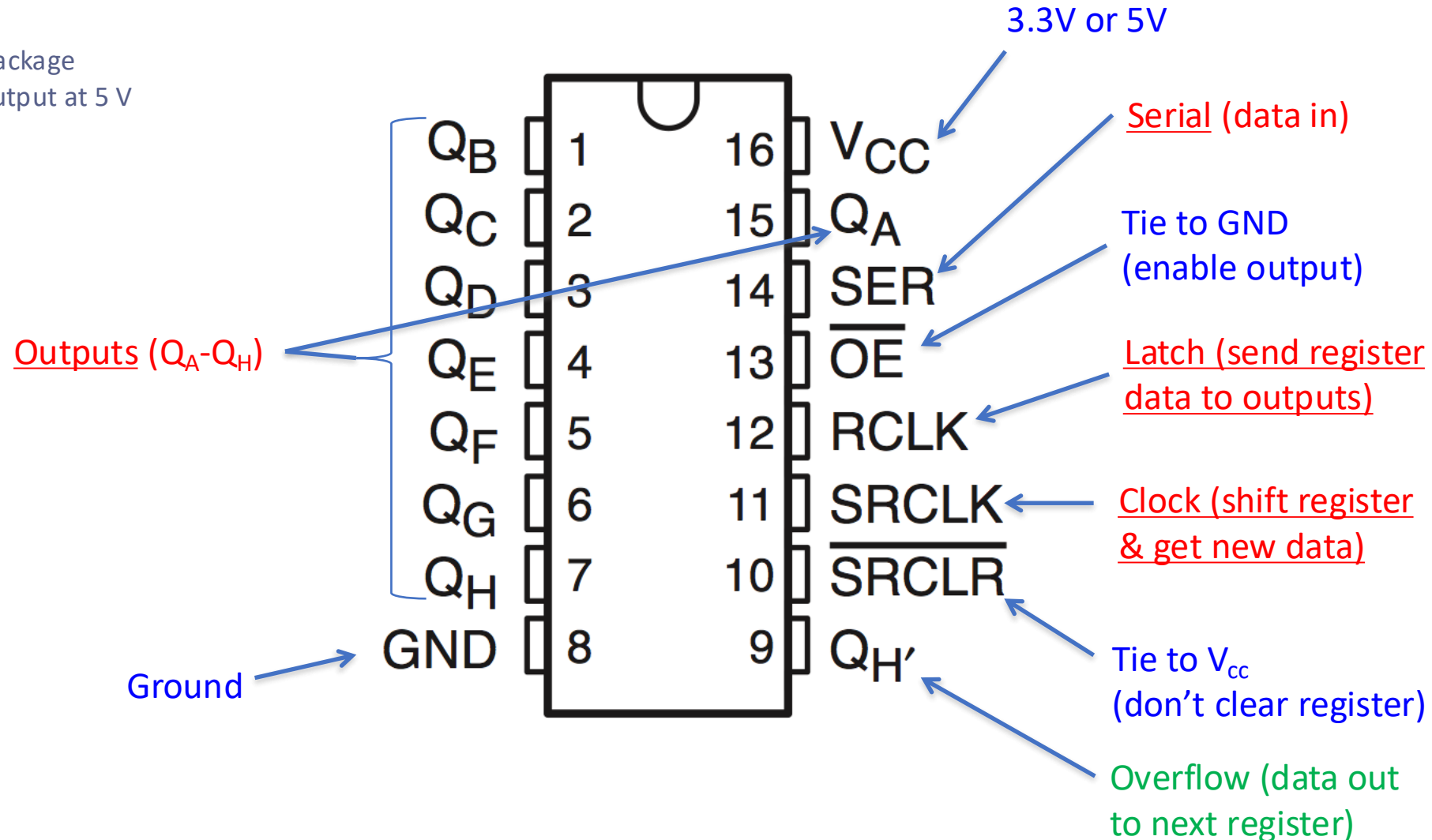
Hardware

Wiring the shift register to the ESP32-C3



SN74HC595 Pinout

DIP-16 package
±6 mA output at 5 V



A bar over a pin name on the datasheet means active LOW: the pin does its job when you pull it to ground.

Datasheet Function Table

SER = serial input (binary data)

SRCLK = shift register clock ("clock")

RCLK = storage register clock ("latch")

INPUTS					FUNCTION
SER	SRCLK	SRCLR	RCLK	OE	
X	X	X	X	H	Outputs Q_A – Q_H are disabled.
X	X	X	X	L	Outputs Q_A – Q_H are enabled.
X	X	L	X	X	Shift register is cleared.
L	↑	H	X	X	First stage of the shift register goes low. Other stages store the data of previous stage, respectively.
H	↑	H	X	X	First stage of the shift register goes high. Other stages store the data of previous stage, respectively.
X	X	X	↑	X	Shift-register data is stored in the storage register.

DECODER

SER	serial input (binary data)
SRCLK	shift register clock — "clock"
RCLK	storage register clock — "latch"
SRCLR	shift register clear
OE	output enable

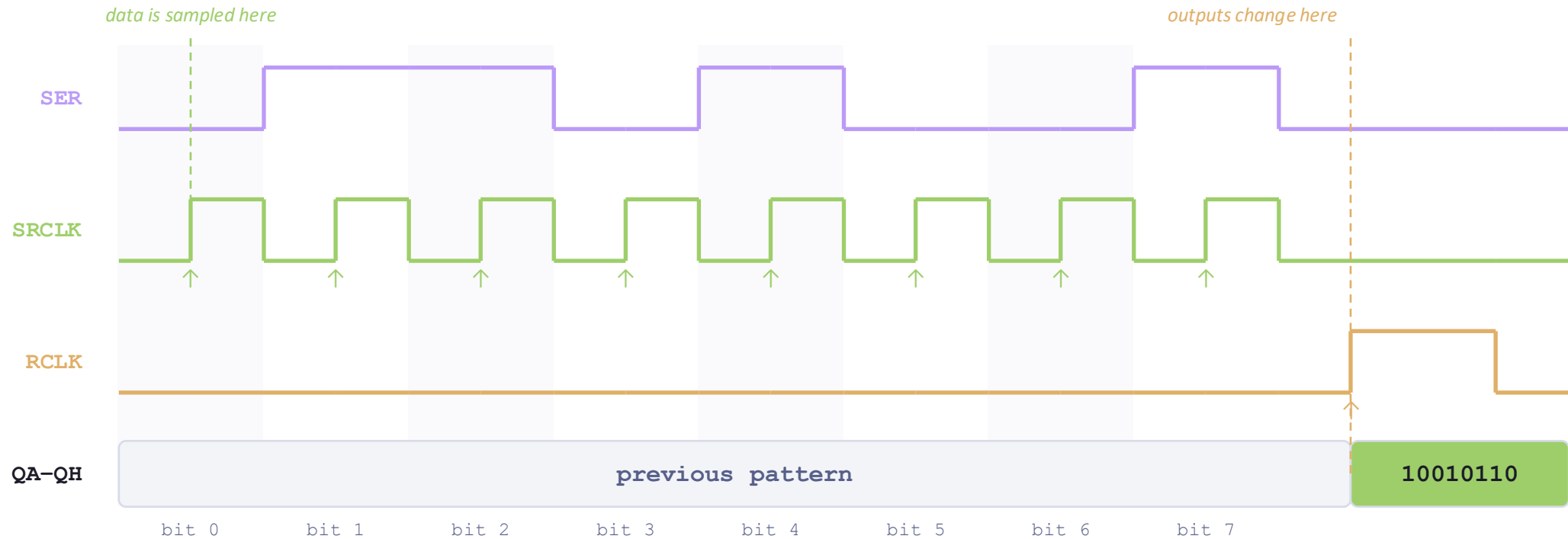
↑ on SRCLK

One bit moves in. Every stage passes its value to the next one, and SER lands in the first stage.

↑ on RCLK

The eight stages are copied to Q_A – Q_H . This is the only row that changes what you can see.

Three Wires Control the Device



- **SER only has to be set at the clock edge**

Between rising edges the data line can be anything. The chip looks at it once per clock and ignores it the rest of the time.

- **SRCLK edges fill the storage register**

A ninth bit would push bit 0 out of QH' and off the end.

- **RCLK edge loads the output register**

The outputs hold the previous byte until the rising RCLK edge.

04

code

Driving the '595 from MicroPython



Shifter class – shifter.py

A class to run a shift register by [bit banging](#).

```
# shifter.py
#
# Shift register class
#
# ESP32-C3 SuperMini

from machine import Pin
import time

class Shifter():

    def __init__(self, data, latch, clock):
        self.dataPin = Pin(data, Pin.OUT)
        self.latchPin = Pin(latch, Pin.OUT)
        self.clockPin = Pin(clock, Pin.OUT)

    def _ping(self, p):
        """ping the clock (CLK) or latch (RCLK) pin"""
        p.value(1)
        time.sleep_us(10)
        p.value(0)
```

- **Three pins, three outputs**

data, latch and clock arguments are pin numbers, not Pin objects. The class turns them into Pins so the caller doesn't need to.

- **`_ping()` is a generic clock signal**

Low, high, low again — one rising edge with reset.

- **Note the leading underscore on `_ping()`**

A Python convention, not a rule: `_ping` is for the class's own use. Nothing stops you calling it, but nothing outside should need to.

The shift() function

shift() presents each bit from an input word, pulses the clock to load each into the register, and pulses the latch once at the end.

```
def shift(self, dataword, num_bits=8):
    """
    Shift all bits in an arbitrary-length word, allowing
    multiple 8-bit shift registers to be chained (with overflow
    of SR<n> tied to input of SR<n+1>)
    """

    # See code file for explanation of why -num_bits%8 works here!
    for i in range(-num_bits%8):          # Load bits short of a byte with 0
        self.dataPin.value(0)
        self._ping(self.clockPin)
    for i in range(num_bits):             # Send the word
        self.dataPin.value(dataword & (1<<i))
        self._ping(self.clockPin)
    self._ping(self.latchPin)
```

- **-num_bits%8 to fill unused bits**

This works because Python's modulo returns an explicitly positive result. See detailed explanation in the code file.

- **dataword & (1<<i) picks ith bit**

Non-zero when that bit is set, and Pin.value() reads any non-zero as HIGH. i counts up from 0, so the low bit goes first.

- **The first loop pads to a byte**

A 12-bit word needs 16 pulses to fill two chips, so four zeros are added in front of the word.

- **One latch at the end**

The outputs stay frozen through every clock pulse and change together on the final _ping().

Different Ways to Build the Byte

```
# Binary value
s.shift(0b10010110)

# Integer value
for i in range(256):
    s.shift(i)

# Bit shifting
pattern = (1 << 0) | (1 << 3) | (1 << 5)    # -> 0b00101001
s.shift(pattern)

# Walk one bit at a time
for i in range(8):
    s.shift(1 << i)

# Set, clear, toggle and test a single bit
pattern |= (1 << 4)      # set bit 4
pattern &= ~(1 << 4)     # clear bit 4
pattern ^= (1 << 4)      # toggle bit 4
if pattern & (1 << 4):   # test bit 4
    print('segment 4 is on')
```

WHICH BIT TO WHICH OUTPUT?

bit 7 → QA

bit 0 → QH



Printing binary words to help debug

`f'{pattern:08b}'` prints the byte as eight padded binary digits – helpful when debugging

Running a Shifter Object

Make a binary counter using 8 LEDs of your 10-segment LED bar.

```
# -----  
# Example  
# -----  
  
if __name__ == "__main__":  
    s = Shifter(data=5, latch=6, clock=7)  
    for i in range(256):  
        s.shift(i)  
        time.sleep(0.1)
```

1

Counting in binary as a test pattern

Every output gets exercised, and a wrong bit order or a missing latch is obvious the moment you look at the bar.

2

Watch which end of the bar moves fastest

QH holds bit 0, so the segment wired to QH toggles on every count. If the fast end is at the other side, your wiring is mirrored.



Try it on your own hardware

Use the 10-segment LED bar and 220 Ω network resistor from the GPIO lab, plus one '595 SR.



If nothing lights up

Check OE is at ground and SRCLR is at V_{CC} before you debug any code. Left floating, those two pins will happily hold the outputs off or wipe the register on you.

1-8 bits = 1 SR, 9-16 bits = 2 SRs , 27-24 bits = 3 SRs, etc.

num_bits defines how many shift registers are being used:

$(N-1)*8 < \text{num_bits} \leq N*8$ implies there are N chained shift registers.

THE CALL	PADDING	PULSES	WHERE THE BITS END UP
<code>shift(v)</code>	0	8	One chip: v. Two chips: v in the near chip, and the near chip's old byte shoved into the far one
<code>shift(v, 10)</code>	6	16	6 zeros travel furthest, then the 10 data bits
<code>shift(v, 16)</code>	0	16	Low byte in the far chip, high byte in the near one



Under-stating can be used to “remember” upstream SR data when new data is shifted in

`shift(v, 8)` on two SRs clocks only 8 pulses, so the 2nd SR it simply inherits whatever byte was previously in the 1st register.

Dimming Everything via PWM on OE Pin

The output enable (OE) pin has been tying to ground until now. Drive it with PWM to control brightness on all outputs at once.

```
from machine import Pin, PWM

# OE is ACTIVE LOW: held low, the outputs are on.
# Drive it with PWM and the outputs spend part of
# every cycle in high impedance -- i.e. off.

oe = PWM(Pin(4), freq=1000)

oe.duty_u16(0)           # always low -> full brightness
oe.duty_u16(32768)       # half           -> about half as bright
oe.duty_u16(58000)       # mostly high -> dim

# Fade the whole bar without touching the pattern
for d in range(0, 65535, 512):
    oe.duty_u16(65535 - d)
    time.sleep_ms(10)
```

- **It is global, not per segment**

Every output dims together, because they all share the one enable pin. Per-LED brightness needs eight PWM channels or a dedicated LED driver.

- **The pattern is untouched**

The shift and storage registers keep their contents the whole time. You are gating the output drivers, not the data.

- **Fast enough by a mile**

The '595 enables and disables its outputs in a few hundred nanoseconds, so a 1 kHz PWM — or 20 kHz, if 1 kHz buzzes — is no strain.

- **It costs a fourth pin**

Three for data, clock and latch, plus this one. Worth it for a display, not worth it for eight indicator LEDs.



Driving Shift Registers via SPI

Let the SPI bus for hardware clocking instead of manual bit-banging.

1

SPI is a shift register protocol

A hardware peripheral that presents a bit on one pin and pulses a clock on another, at MHz rates, without Python in the loop.

2

2 of the 3 pins come for free

mosi carries the data and sck carries the clock. Only the latch stays an ordinary GPIO you toggle yourself.

3

The bus is shareable

Displays, SD cards and sensors can sit on the same sck and mosi lines. Each device just needs its own latch or chip-select pin.

Same three wires on the chip. SER goes to the SPI mosi pin, SRCLK to sck, and RCLK to whatever GPIO you hand the class as latch.

Setting up the SPI object

```
spi = SPI(  
    1,  
    baudrate=1000000,  
    polarity=0,  
    phase=0,  
    sck=Pin(7),  
    mosi=Pin(5)  
)
```

polarity=0, phase=0

SPI mode 0: the clock idles LOW and data is sampled on the rising edge — which is precisely what the '595 expects on SRCLK. The other three modes will not work here.



shifter_spi.py

```
# shifter_spi.py
#
# Shift register class using hardware SPI
# Supports one or more daisy-chained 74HC595 shift registers.
#
# ESP32-C3 SuperMini

from machine import Pin, SPI
import time

class Shifter:

    def __init__(self, spi, latch):
        """
        spi    : Pre-initialized machine.SPI object
        latch  : GPIO connected to STCP (RCLK) of the 74HC595
        """
        self.spi = spi
        self.latch = Pin(latch, Pin.OUT)
        self.latch.off()
```

- **The SPI object is passed in, not built**

The class does not own the bus. Whoever creates the SPI decides the pins and the baud rate, and other devices can share it.

- **Only the latch is a pin now**

data and clock have disappeared from the constructor — the SPI peripheral owns them.

- **latch.off() before anything else**

RCLK must start LOW so the first pulse is a rising edge.



Matching shifter_spi.py and shifter.py

SPI hardware clocks each byte out MSB-first. shifter.py sends bit 0 first. One small table reconciles them.

MODULE LEVEL — THE BIT-ORDER FIX

```
def _rev8(b):
    b = ((b & 0x0F) << 4) | (b >> 4)
    b = ((b & 0x33) << 2) | ((b & 0xCC) >> 2)
    b = ((b & 0x55) << 1) | ((b & 0xAA) >> 1)
    return b

_REV = bytes(_rev8(i) for i in range(256))

def _lsb_first(data):
    return bytes(_REV[b] for b in data)
```

IN THE CLASS — UNCHANGED

```
def _latch(self):
    """ Transfer shifted data to the output pins """
    self.latch.on()
    time.sleep_us(1)
    self.latch.off()
```

● The peripheral sends bit 7 first

shifter.py presents bit 0 first, so without help the same integer lights the bar backwards. Reversing every byte on the way out is what makes the two classes agree.

● Why not firstbit=SPI.LSB?

That flips the whole bus. Any display, SD card or sensor sharing sck and mosi expects MSB-first — reversing inside the class leaves them alone.

● 256 bytes, built once at import

_REV[b] is b with its bits reversed. The table is its own inverse, so the same call undoes itself if you ever read data back in.



shift() and write_bytes()

The same two public methods as shifter.py, with the three lines that put the bits in the same order.

```
def shift(self, value, num_bits=8):
    """
    Shift an arbitrary-length word to support chained registers
        value      : integer to transmit
        num_bits   : number of valid bits
    """
    nbytes = (num_bits + 7) // 8
    value &= (1 << num_bits) - 1      # Mask off unused upper bits
    value <= -num_bits % 8            # Pad short of a byte with 0
    self.spi.write(_lsb_first(value.to_bytes(nbytes, 'little')))
    self._latch()

def write_bytes(self, *values):
    """
    Write one or more bytes, one per register in the chain.
    The FIRST argument travels furthest.
    """
    self.spi.write(_lsb_first(bytes(values)))
    self._latch()
```

- **Mask, pad, then reverse**

Three changes: mask to num_bits, pad in the LOW bits, and send little-endian with each byte reversed. Drop any one and the pattern comes out mirrored.

- **spi.write() clocks out every bit**

The peripheral drives mosi and sck together, one bit per clock, without Python in the loop.

- **write_bytes() is one byte per chip**

The first argument is sent first, so it ends up furthest away: write_bytes(lo, hi) is the same as shift(value, 16).



_latch()

One call into the SPI peripheral, one latch pulse. The bit loop is gone entirely.

```
def _latch(self):  
    """ Transfer shifted data to the output pins """  
    self.latch.on()  
    time.sleep_us(1)  
    self.latch.off()
```

- **_latch() replaces _ping()**

The clock is no longer your problem — only the latch is. One microsecond is plenty of hold time for RCLK.



Running It Over SPI

The same counter as before. The only thing that changed is who toggles the clock.

```
if __name__ == "__main__":

    # Create the SPI object for use by Shifter
    spi = SPI(
        1,
        baudrate=1000000,
        polarity=0,
        phase=0,
        sck=Pin(7),
        mosi=Pin(5)
    )

    s = Shifter(spi, latch=6)

    while True:
        for i in range(256):
            s.shift(i)
            time.sleep(0.1)
```

- **Build the bus, then the device**

The SPI object is created once and handed to Shifter. A second peripheral on the same bus gets the same object and its own chip-select.

- **sck is SRCLK, mosi is SER**

Those two pins go straight to the chip. GPIO 6 stays the latch, exactly as it was in the bit-banged version.

- **Identical call, identical result**

s.shift(i) is the same method name, the same default width and now the same bit order as shifter.py. Swapping the classes changes nothing you can see.

- **Only the timing is different**

time.sleep(0.1) still sets the pace. Without it, the counter would run far faster than your eye can follow.



How Fast using SPI?

DATASHEET LIMIT	AT 2 V	AT 4.5 V	WHAT IT MEANS FOR US
Clock frequency, max	5 MHz	25 MHz	1 MHz SPI is 5× inside the worst case
SRCLK / RCLK pulse width, min	100 ns	20 ns	_ping() holds 10 μs — 100× longer than needed
SER setup before SRCLK ↑	125 ns	25 ns	SPI presents each bit half a clock early
SER hold after SRCLK ↑	0 ns	0 ns	You cannot be too late letting go of the data
RCLK ↑ to outputs valid	187 ns	37 ns	The LEDs are lit before your next line runs

There is no 3.3 V column

The datasheet characterises the part at 2 V, 4.5 V and 6 V. At 3.3 V you are between columns, so design against the 2 V numbers and you cannot be wrong.

A footnote worth knowing

"The clocks can be tied together, in which case the shift register is one clock pulse ahead of the storage register." Two wires instead of three — at the cost of the outputs always trailing by one bit.



Which One Should You Use?

Both classes put the same bits on the same three wires, in the same order. They differ in who does the clocking.

	<code>shifter.py</code> — bit-banged	<code>shifter_spi.py</code> — hardware SPI
Pins	Any three GPIO you like	The SPI peripheral's sck and mosi, plus one latch GPIO
Clocking	Python toggles the clock, ~10 μ s held per edge	The peripheral clocks bits out at the baud rate you set
Speed	Roughly a millisecond per byte	A byte at 1 MHz is 8 μ s — finished before the next statement
Bit order	Bit 0 goes out first, so QH holds the low bit	Bit 0 first as well — each byte is reversed on the way out to match
Reach for it when	You want to watch every edge, or the SPI pins are spoken for	You are driving chains, displays, or anything on a schedule

Same method name, same default width, same bit order — so `shift()` reads identically either way. Start bit-banged while the wiring is unproven, since you can slow the clock down and step through it; move to SPI once you trust the hardware and want the CPU back.

Lab 5 - Shift Registers

Individual work



Lab 6 – Cascaded Shift Registers

Team-based work (3-4 students per team)



Next Steps



Concurrency

`_thread`, `asyncio`, and revisiting IRQs.



Stepper Motors

Driving stepper motors using a Darlington array.